

บทที่ 6

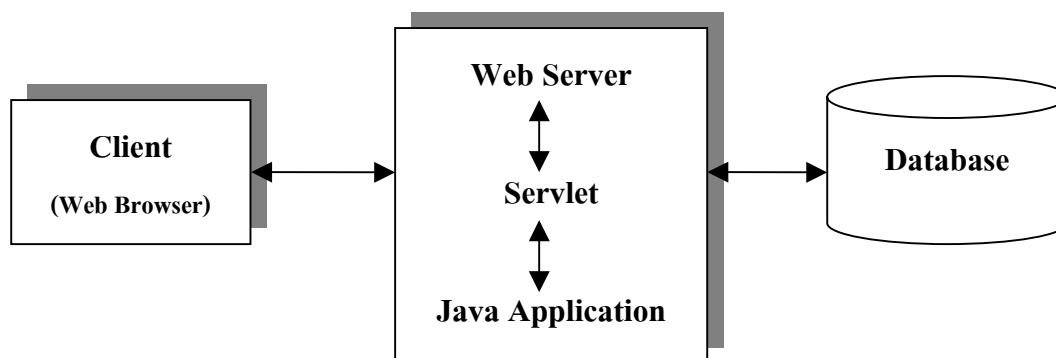
จาวาเซิร์ฟเลต (Java Servlet)

แม้ว่าโลกของอินเทอร์เน็ตจะเพิ่งเกิดขึ้นเพียงไม่กี่ปีก็ตาม แต่เทคโนโลยีที่ใช้กับอินเทอร์เน็ตกลับมีการเปลี่ยนแปลงไปอย่างรวดเร็วมาก ในสมัยแรกๆ หน้าเพจต่างๆ ที่อยู่ในเว็บจะเป็นลักษณะของสแตติกเพจหรือเพจที่ไม่มีการเปลี่ยนแปลงเนื้อหาที่ไม่ว่าจะนานเท่าไร นอกเสียจากว่าผู้ดูแลเพจนั้นจะทำการอัปเดตเพจดังกล่าว เพจลักษณะนี้เป็นเพจที่นิยมใช้กันทั่วไปในอินเทอร์เน็ตสมัยแรกเพราะอินเทอร์เน็ตยังนิยมกันอยู่ในวงแคบ โดยกลุ่มผู้ใช้งานจะเป็นกลุ่มบุคคลที่อยู่ในวงการศึกษาเท่านั้น ต่อมาจากนั้นไม่นานทางผู้ผลิตบราวเซอร์ได้ทำการเพิ่มความสามารถให้กับเพจโดยอนุญาตให้เพจสามารถแทรกสคริปต์เล็กๆ ลงไปรวมกับส่วนที่เป็น HTML ได้ซึ่งจุดนี้ก็คือจุดเริ่มต้นของจาวาสคริปต์ทางฝั่งไคลเอนต์นั่นเอง

แม้ว่าเพจจะเริ่มมีความสามารถในการโต้ตอบกับผู้ใช้โดยอิงความสามารถจากจาวาสคริปต์แล้วก็ตาม ถ้าพูดถึงในแง่ของส่วนเนื้อหาของตัวเพจจริงๆ แล้วตัวเพจเองก็ยังคงเป็นเพจสแตติก อยู่เช่นเดิม เมื่อกลุ่มผู้ใช้อินเทอร์เน็ตเริ่มมีมากขึ้นความต้องการที่จะทำให้เพจสามารถทำการรับส่งข้อมูลรวมไปถึงการเปลี่ยนแปลงเนื้อหาได้โดยอัตโนมัติก็เกิดขึ้น เทคโนโลยีที่เกิดขึ้นเพื่อรองรับความต้องการเหล่านี้ก็คือแอปพลิเคชันทางฝั่งเซิร์ฟเวอร์ (Server Side Application) นั่นเอง

6.1 ความหมายของเซิร์ฟเลต

เซิร์ฟเลตเป็นแอปพลิเคชันที่อยู่ทางฝั่งเซิร์ฟเวอร์แบบหนึ่งซึ่งอ้างอิงคอนเซ็ปต์มาจาก CGI (Common Gateway Interface) ข้อดีของเซิร์ฟเลตที่อยู่เหนือ CGI อย่างแรกก็คือตัวภาษาที่ใช้เขียนซึ่งก็คือภาษาจาวานั้นเอง จาวาเป็นภาษาที่ใช้คอนเซ็ปต์ของออบเจกต์-โอเรียนเต็ด (Object-Oriented) การเขียนโปรแกรมแบบออบเจกต์-โอเรียนเต็ด สามารถลดความซับซ้อนของโครงสร้างโปรแกรมรวมไปถึงการอำนวยความสะดวกในการนำส่วนของโปรแกรมที่เขียนไว้แล้วมาใช้ใหม่ได้ นอกจากนี้จาวายังเป็นภาษาที่เป็นลักษณะแบบไม่ขึ้นกับแพลตฟอร์มซึ่งจะช่วยให้เราสามารถที่จะทำการพัฒนาระบบโดยใช้สภาพแวดล้อมอะไรก็ได้ซึ่งโดยทั่วไปมักนิยมใช้ Window โดยจะนำโปรแกรมที่เขียนเสร็จแล้วมารันบนยูนิกซ์ เพื่อเพิ่มความเสถียรภาพของโปรแกรมแทน นอกจากนี้เซิร์ฟเลตยังมีความเร็วที่สูงกว่า CGI เพราะเซิร์ฟเลตใช้หลักการของเทรด (thread) โดยทำการสร้างหนึ่งเทรดต่อหนึ่งคำร้องขอที่มาจากไคลเอนต์ซึ่งในทางกลับกัน CGI จะทำการสร้างหนึ่งโพรเซส (process) ต่อหนึ่งคำร้องขอ ซึ่งจะทำให้เปลืองทรัพยากรมากกว่า และโพรเซสในการรันก็จะช้ากว่าด้วย ท้ายที่สุดจุดเด่นที่สำคัญของเซิร์ฟเลตก็คือ API (Application Programming Interface) โดยระบบที่ทำการพัฒนาโดยใช้คอนเซ็ปต์ของเซิร์ฟเลตจะสามารถเรียกใช้ API ที่ทางจาวามีมาให้ซึ่งจะช่วยทำให้การพัฒนาระบบดังกล่าวง่ายและเร็วยิ่งขึ้น

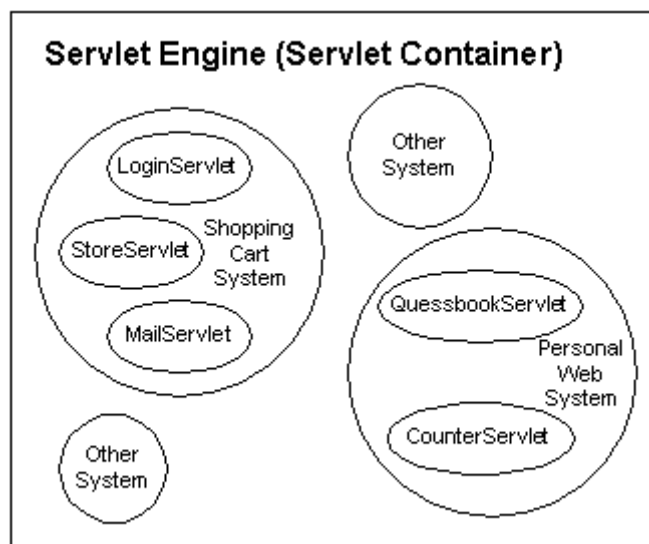


รูปที่ 6-1 หน้าที่และการทำงานของเซิร์ฟเล็ต

เมื่อมีการขอเรียกใช้บริการเซิร์ฟเล็ตมาถึงยังเซิร์ฟเวอร์ เซิร์ฟเล็ตเอนจินจะเรียกตัวโหลดคลาส(class loader) ตัวโหลดคลาสจะทำการตรวจสอบว่าคลาสดังกล่าวถูกโหลดขึ้นมาเรียบร้อยแล้วหรือไม่ และตรวจสอบเวอร์ชันของคลาสที่ถูกโหลดว่ามีค่าไทม์แสตมป์ (time stamp) เท่ากับ .class ไฟล์ (.class file) ที่อยู่บนดิสก์ (disk) หรือไม่ ถ้าคลาสของเซิร์ฟเล็ตยังไม่ถูกโหลด หรือมีเวอร์ชันเก่ากว่าไฟล์ที่อยู่บนดิสก์ เซิร์ฟเล็ตเอนจินจะทำการโหลดคลาสไปเก็บไว้ใน JVM และสร้างอินสแตนซ์ (instance) ของคลาสดังกล่าว หลังจากนั้นวงจรชีวิตของเซิร์ฟเล็ตจะเริ่มขึ้นใหม่

6.2 เซิร์ฟเล็ตเอนจิน (Servlet engine)

ในการรันระบบที่เขียนขึ้นโดยใช้หลักการของเซิร์ฟเล็ตเราจะต้องนำระบบดังกล่าวมาบรรจุอยู่ในสิ่งๆ หนึ่งที่เรียกว่าเซิร์ฟเล็ตเอนจิน ให้นึกว่าเซิร์ฟเล็ตเอนจินคล้ายๆกับกล่องๆหนึ่งที่ใส่ลูกปิงปองไว้หลายลูก โดยลูกปิงปองแต่ละลูกก็คือระบบๆ หนึ่งนั่นเอง หลายคนอาจสงสัยทำไมถึงใช้คำว่าระบบ โดยทั่วไปแอปพลิเคชันทางฝั่งเซิร์ฟเวอร์ต่างๆ ที่ถูกเขียนขึ้นโดยใช้ API ของเซิร์ฟเล็ตจะถูกเรียกว่าเซิร์ฟเล็ต ในหนึ่งระบบอาจประกอบด้วยเซิร์ฟเล็ตหลายอัน ยกตัวอย่างเช่น ระบบที่เกี่ยวกับ Shopping Cart อาจจะประกอบด้วยเซิร์ฟเล็ตที่ทำหน้าที่ในการเช็คสต็อกอิน, เซิร์ฟเล็ตที่ทำหน้าที่ในการเก็บข้อมูลสินค้า, เซิร์ฟเล็ตที่ทำหน้าที่ในการส่งเมลกลับไปยังลูกค้าเพื่อบอกว่าได้ทำการส่งของไปให้แล้ว เป็นต้น ดังนั้นถ้ามองโดยรวมแล้วเซิร์ฟเล็ตเอนจินก็คือที่รวมของระบบตั้งแต่หนึ่งระบบถึงหลายระบบ โดยแต่ละระบบจะประกอบด้วยเซิร์ฟเล็ตหนึ่งอันหรือมากกว่า ดังรูปที่ 6-2 ซึ่งระบบในที่นี้อาจจะหมายถึง Zone (Apache Jserv) หรือ Web Application (Tomcat) ก็ได้



รูปที่ 6-2 เซิร์ฟเลตเอนจิน และเซิร์ฟเลตที่อยู่ภายใน

เซิร์ฟเลตเอนจินเป็นเพียงกล่องๆหนึ่งที่ใช้บรรจุและรันกลุ่มของเซิร์ฟเลตเท่านั้น ในการที่จะทำการติดต่อสื่อสารกับไคลเอนต์ ตัวเซิร์ฟเลตเอนจินนี้จะต้องทำงานร่วมกับเว็บเซิร์ฟเวอร์ ซึ่งเปรียบเสมือนฉากหน้าที่ติดต่อกับไคลเอนต์อีกทีหนึ่ง เมื่อใดก็ตามที่มีคำร้องขอส่งมาจากไคลเอนต์ ถ้าคำร้องขอนั้นจะจงมาที่ตัวเซิร์ฟเลต ทางเว็บเซิร์ฟเวอร์ก็จะทำการส่งคำร้องขอนั้นมาให้เซิร์ฟเลตเอนจิน ซึ่งทางเซิร์ฟเลตเอนจินก็จะทำการเรียกเซิร์ฟเลตที่ไคลเอนต์ต้องการขึ้นมาทำการประมวลผลคำร้องขอนั้น โดยท้ายสุดเซิร์ฟเลตจะส่งผลกลับไปให้เซิร์ฟเลตเอนจิน เซิร์ฟเลตเอนจินก็จะส่งผลที่ได้กลับไปให้เว็บเซิร์ฟเวอร์ ซึ่งเว็บเซิร์ฟเวอร์ก็จะส่งผลกลับไปให้ไคลเอนต์

เซิร์ฟเลตเอนจินอาจจะเป็นส่วนที่ติดมากับเว็บเซิร์ฟเวอร์อยู่แล้วยกตัวอย่างเช่น เซิร์ฟเลตเอนจินที่อยู่ในเซิร์ฟเวอร์ของ Netscape Enterprise, IBM Web Sphere หรืออาจจะเป็นส่วนที่เป็นส่วนที่เพิ่มเติมให้กับเว็บเซิร์ฟเวอร์ก็ได้เช่น Apache Jserv, Tomcat, JRun หรือแม้กระทั่งเป็นส่วนหนึ่งที่อยู่ในเว็บแอปพลิเคชัน เช่น BEA Web logic เป็นต้น ทั้งนี้การเลือกใช้เซิร์ฟเลตเอนจินแต่ละชนิดก็มักขึ้นอยู่กับปัจจัยหลายอย่างเช่น ความสะดวกในการรวมระบบที่จะสร้างขึ้นมาใหม่กับระบบที่มีอยู่แล้ว, งบประมาณที่มีอยู่สำหรับโครงการหรืออาจจะรวมไปถึงทักษะและประสบการณ์ส่วนตัวของนักพัฒนาแต่ละคน

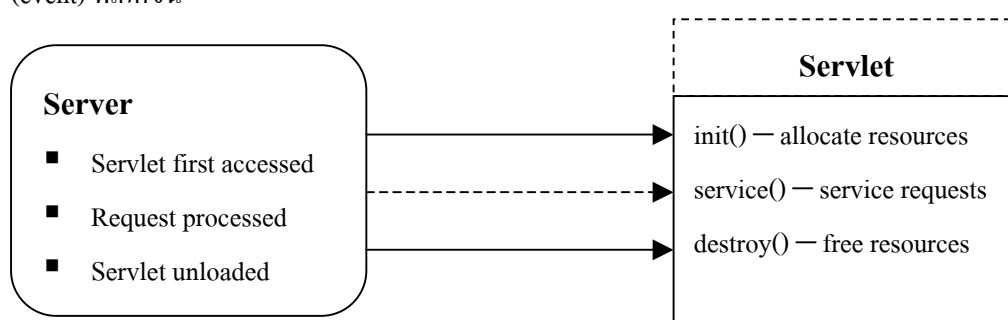
6.3 อินเทอร์เฟซ javax.servlet.Servlet

หัวใจของเซิร์ฟเลตจริงๆอยู่ที่อินเทอร์เฟซที่ชื่อ javax.servlet.Servlet* โดยทุกเซิร์ฟเลตที่ถูกเขียนขึ้นจะต้องทำการอิมพลีเมนต์ตัวอินเทอร์เฟซนี้ไม่ทางตรงก็ทางอ้อม (ทางตรงก็คือการอิมพลีเมนต์ตัวอินเทอร์เฟซนี้เลย ส่วนทางอ้อมก็คือการให้เซิร์ฟเลตทำการแบ่งย่อยคลาสบางคลาสที่ได้ทำการอิมพลีเมนต์ตัวอินเทอร์เฟซนี้ไว้แล้ว) เหตุผลว่าทำไมเราต้องอิมพลีเมนต์ตัวอินเทอร์เฟซนี้เพราะว่าเมื่อไรก็ตามที่มีคำร้องขอจากไคลเอนต์เข้ามายังเซิร์ฟเลตเอนจิน ตัวเซิร์ฟเลตเอนจินจะทำการหาเซิร์ฟเลตที่ทำการร้องขอดังกล่าวอ้างถึง หลังจากนั้นเซิร์ฟเลตเอนจินจะทำการเรียกฟังก์ชันต่างๆที่อยู่ในเซิร์ฟเลตเพื่อทำการประมวลผลคำ

ร้องขอของไคลเอนต์ โดยฟังก์ชันที่เซิร์ฟเลตเอนจินจะทำการเรียกก็คือฟังก์ชันที่เซิร์ฟเลตได้ทำการอิมพลิเมนต์ ซึ่งเป็นฟังก์ชันที่ถูกกำหนดอยู่ใน `javax.servlet.Servlet` อินเตอร์เฟสนั่นเอง อาจมีคำถามว่าทำไมเซิร์ฟเลตเอนจินถึงเรียกฟังก์ชันที่ถูกกำหนดอยู่ในอินเตอร์เฟสนี้ เหตุผลง่าย ๆ ก็คือเซิร์ฟเลตเป็นส่วนที่ถูกโหลดเข้าไปในเซิร์ฟเลตเอนจินในช่วง Runtime ตัวเซิร์ฟเลตเอนจินเองไม่สามารถทราบได้ว่าเซิร์ฟเลตต่าง ๆ มีฟังก์ชันอะไรประกอบอยู่บ้างนอกเสียจากว่าเซิร์ฟเลตนั้นได้ทำการอิมพลิเมนต์ฟังก์ชันที่เป็นมาตรฐานที่เซิร์ฟเลตเอนจินรับรู้ ซึ่งนี่ก็คือเหตุผลว่าทำไมทุกเซิร์ฟเลตจะต้องทำการอิมพลิเมนต์ตัว `javax.servlet.Servlet` อินเตอร์เฟส อย่างไรก็ตามเราสามารถให้เซิร์ฟเลตเอนจินเรียกฟังก์ชันอื่นๆที่อยู่ในเซิร์ฟเลตได้ ซึ่งวิธีการก็คือการใส่ฟังก์ชันดังกล่าวเข้าไปในส่วนอิมพลิเมนต์ของฟังก์ชันต่างๆที่ถูกกำหนดอยู่ใน `javax.servletServlet` อินเตอร์เฟส

6.4 วงจรชีวิตของเซิร์ฟเลต (Servlet's Life Cycle)

วงจรชีวิตของเซิร์ฟเลตจะประกอบด้วย 3 สถานะคือ สถานะเริ่มต้น (`init()`), สถานะให้บริการ(`service()`) และสถานะทำลาย (`destroy()`) เมธอดเหล่านี้เป็น “Callback method” หมายถึงเมธอดจะไม่ถูกเรียกโดยตัวเซิร์ฟเลตเอง แต่จะถูกเรียกโดยเซิร์ฟเลตเอนจิน ในการตอบสนองกับเหตุการณ์ (event) ที่เกิดขึ้น



รูปที่ 6-3 วงจรชีวิตของเซิร์ฟเลต

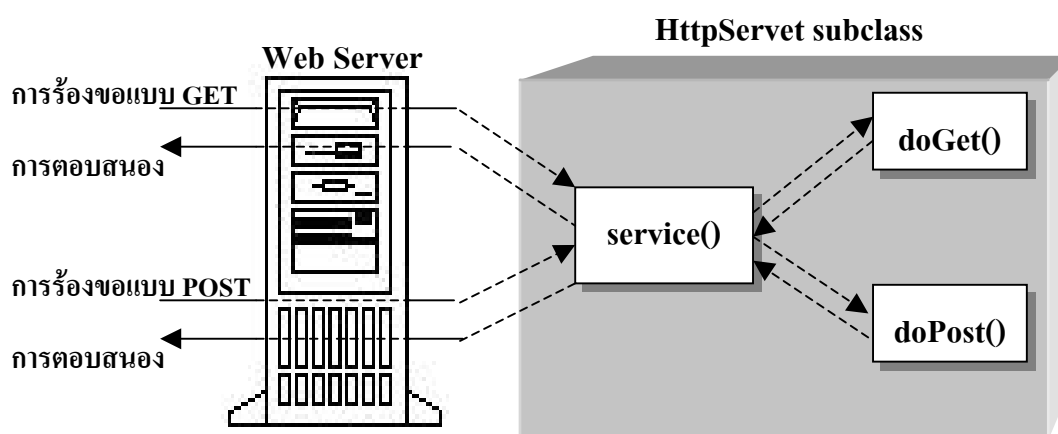
6.4.1 สถานะเริ่มต้น (Initial State)

เป็นการใช้เมธอด `init()` ซึ่งเมธอดจะถูกเรียกโดยเซิร์ฟเวอร์ทันทีที่อินสแตนซ์ (instance) ถูกสร้างเสร็จเรียบร้อยแล้ว โดยปกติเมธอดนี้จะถูกใช้เพื่อทำการเริ่มต้นเซิร์ฟเลต สาเหตุที่ไม่ใช้คอนสตรัคเตอร์ (constructor) เพราะคอนสตรัคเตอร์ของ JDK 1.0 (เริ่มมีเซิร์ฟเลต) ไม่สามารถรับอาร์กิวเมนต์ (argument) ในกรณีที่เกิดการโหลดแบบไดนามิก (dynamic) ได้ ดังนั้นการที่จะจัดหาข้อมูลเกี่ยวกับตัวมันเองและสิ่งแวดล้อมให้แก่เซิร์ฟเลตที่ถูกโหลดเป็นครั้งแรก เซิร์ฟเวอร์จำเป็นต้องเรียกเมธอด `init()` และส่งผ่านออบเจกต์ (object) ที่ใช้อินเตอร์เฟส `ServletConfig` นอกจากนี้ภาษาจาวาไม่อนุญาตให้อินเตอร์เฟสมีการประกาศคอนสตรัคเตอร์ นั่นหมายถึงอินเตอร์เฟสของแพ็คเกจ (package) `javax.servlet.Servlet` ก็ไม่สามารถประกาศคอนสตรัคเตอร์ที่รับค่าพารามิเตอร์ (parameter) `ServletConfig` ได้เช่นกัน จึงต้องประกาศเมธอดชื่ออื่นแทนเช่น `init()` แต่ยังคงมีทางเป็นไปได้ที่จะประกาศคอนสตรัคเตอร์ให้แก่เซิร์ฟเลต แต่ใน

คอนสตรัคเตอร์ดังกล่าวห้ามมีการเข้าถึงออบเจกต์ ServletConfig และไม่สามารถ throw ServletException ได้ ออบเจกต์ ServletConfig จะส่งพารามิเตอร์ที่จำเป็นต้องใช้ในการเริ่มต้นการทำงานให้แก่เซิร์ฟเลต พารามิเตอร์ดังกล่าวจะถูกป้อนให้แก่เซิร์ฟเลต โดยไม่เกี่ยวข้องกับการร้องขอที่เข้ามา

6.4.2 สถานะให้บริการ (Service State)

สถานะให้บริการจะถูกเรียกจากเซิร์ฟเวอร์ทุกครั้งที่ไคลเอนต์ทำการร้องขอใช้บริการเซิร์ฟเลต เมธอด service() จะรับค่าพารามิเตอร์ 2 ค่าได้แก่ออบเจกต์การร้องขอและออบเจกต์การตอบสนอง ออบเจกต์การร้องขอจะบอกเซิร์ฟเลตเกี่ยวกับการขอใช้บริการที่มีเข้ามา ในขณะที่ออบเจกต์การตอบสนองจะถูกเรียกใช้ในการส่งการตอบรับคืนกลับให้แก่เซิร์ฟเวอร์ โดยปกติแล้วเมธอด service() จะไม่ถูกโอเวอร์ไรด์ (override) แต่จะทำการโอเวอร์ไรด์เมธอด doGet() เพื่อใช้จัดการการร้องขอแบบ GET และเมธอด doPost() สำหรับจัดการกับการร้องขอแบบ POST แทน ดังตัวอย่างในรูป



รูปที่ 6-4 วิธีการจัดการกับร้องขอแบบ GET และ POST ของเซิร์ฟเลต

6.4.3 สถานะทำลาย (Destroy State)

สถานะทำลายเซิร์ฟเวอร์จะเรียกเมธอด destroy() ของเซิร์ฟเลต เมื่อเซิร์ฟเลตกำลังจะถูกอันโหลด (unload) ในเมธอด destroy() ควรมีการคืนทรัพยากรต่างๆกลับคืนมาให้แก่ระบบ เพื่อให้โปรแกรมอื่นๆจะได้นำทรัพยากรเหล่านี้ไปใช้ได้ นอกจากนี้เมธอดยังให้โอกาสเซิร์ฟเลตในการบันทึกสถานะของข้อมูลต่างๆที่ถูกเรียกใช้เป็นประจำจากการเรียกเมธอด init() ในครั้งต่อไป